

L2 - 3.3. Elastic: Using Elastic Defend

Task 1 - Introduction

The endpoint security component of the Elastic Security suite, enabling detection, investigation, and response directly on monitored hosts.

- Username: `elastic`
- Password: `w4h*Pn=7+WtUUmixs03H`

Task 2 - Configuring Elastic Defend

Elastic Defend protects endpoints by continuously collecting system telemetry to detect suspicious behavior, block threats in real time, and deliver actionable insights for security investigations.

What Does Elastic Defend Protect? : Elastic Defend provides comprehensive endpoint and cloud protection by combining next-generation antivirus, behavioral monitoring, and real-time detection and response across desktops, servers, containers, and Kubernetes clusters.

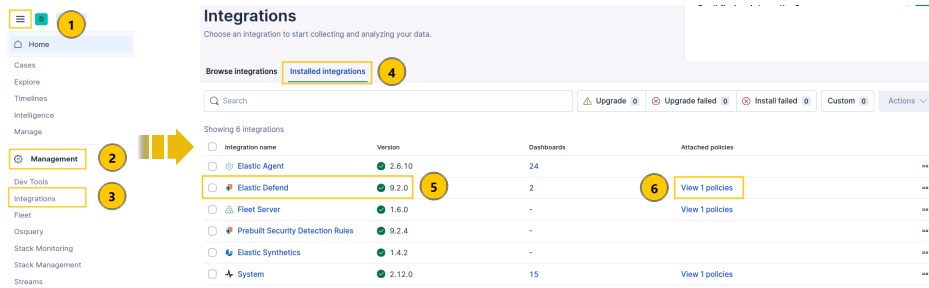
How Elastic Defend Works: Elastic Defend has two distinct capabilities: endpoint telemetry collection and endpoint protection.

Defend Capability	Use Case
Telemetry Collector (Event Visibility)	Collects and forwards endpoint activity (process, file, network) for visibility and investigation, without taking preventive action
NGAV / EDR (Prevention + Detection)	Provides malware detection, ransomware and memory protection, credential theft defense, and the ability to prevent or block malicious activity in real time

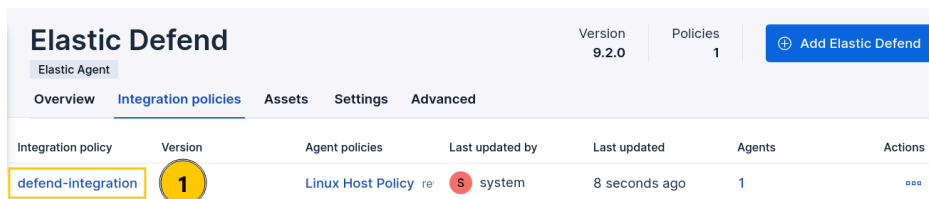
What Does Elastic Defend Protect Against?: Elastic Defend safeguards endpoints by monitoring real-time system behavior—including processes, files, memory, and network activity—to detect and prevent both known malware and advanced zero-day threats.

Configuring Elastic Defend

1. Select the Kibana menu in the top left
2. Scroll down to `Management`
3. Select `Integrations`
4. Click `Installed integrations`
5. Locate `Elastic Defend`
6. Click `View 1 policies`



1. Click the integration policy `defend-integration`, and let's check it out



If you head to the `Malware` section, you can toggle the feature on or off and set the protection level.

- **Detect:** Elastic monitors, an alert is generated if malicious activity is detected, but nothing is blocked
- **Prevent:** Elastic monitors, an alert is generated if malicious activity is detected, and Elastic Defend takes action by quarantining, blocking, or terminating processes



Lastly, head down to the `Event Collection Linux` and `Session data` sections. By default, Elastic Defend will collect file, network, and process data. Collecting session data allows us to track process trees and reconstruct full attack chains during investigations, which we will explore later. Select the two options below and save.

- `Collect session data`
- `Capture terminal output`

Type

Operating system

3 / 3 event collections enabled

Event collection

Linux

Events

☒ File
 ☒ Network
 ☒ Process

Telemetry Data Collected

Session data

Turn this on to capture the extended process data required for Session View. Session View provides you a visual representation of session and process execution data. Session View data is organized according to the Linux process model to help you investigate process, user, and service activity on your Linux infrastructure.

☒ Collect session data
 ☒ Capture terminal output ?

Select and Save

Answer the questions below

What version of the Elastic Defend integration is installed on your Linux host machine?

9.2.0

Which default protection level does Elastic Defend use for Malware?

Prevent

Task 3 - Logging With Elastic Defend

In this task, we'll explore what Elastic Defend monitors, how that data is structured within Elasticsearch, and how to investigate using Discover.

Type

Operating system

Event collection

Windows

Events

☒ API
 ☒ DLL and Driver Load
 ☒ DNS
 ☒ File
 ☒ Network
 ☒ Process
 ☒ Registry
 ☒ Security

Windows

Type

Operating system

Event collection

Mac

Events

☒ DNS
 ☒ File
 ☒ Process
 ☒ Network
 ☒ Security

Mac

Type

Operating system

Event collection

Linux

Events

☒ File
 ☒ Network
 ☒ Process

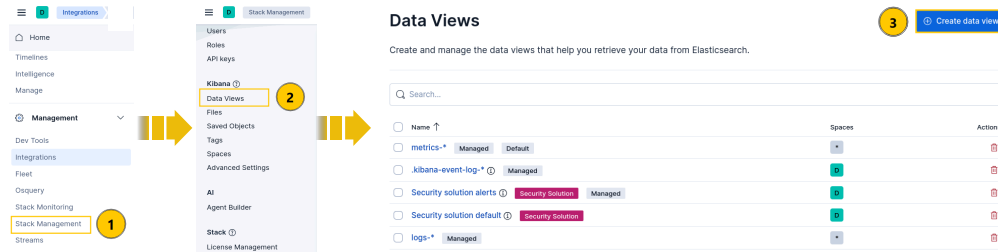
Linux

Elastic Defend Logging Capabilities

Monitoring Type	What It Captures	Key Fields
File Activity	File creation, modification, deletion	File paths, names, hashes, sizes, and associated processes
Network Activity	Outbound connections, IPs, ports, protocols	Source and destination IP addresses, ports, protocols, associated processes, and connection direction
Process Activity	Process starts/stops, command lines, parent-child relationships	Process name, command line, parent process, execution path, and user context

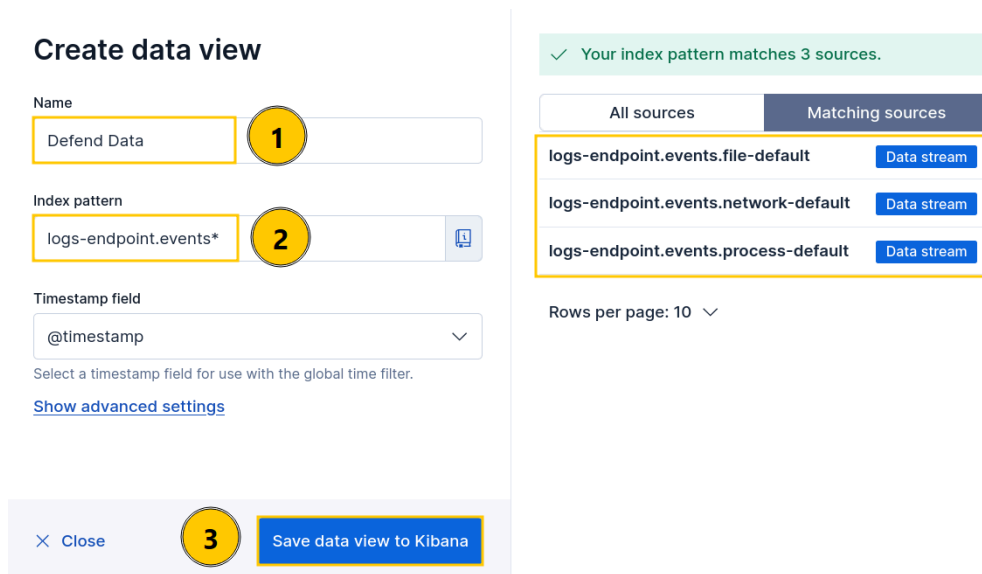
Building an Elastic Defend Data View

1. Scroll down to **Stack Management**
2. Select **Data Views** in the **Kibana** section
3. Choose **Create data view**



In the **Create data view** flyout panel.

1. Give your data view a name **Defend Data**
2. Enter the Index pattern **logs-endpoint.events***, which covers **file**, **network**, and **process** events
3. Click **Save data view to Kibana**



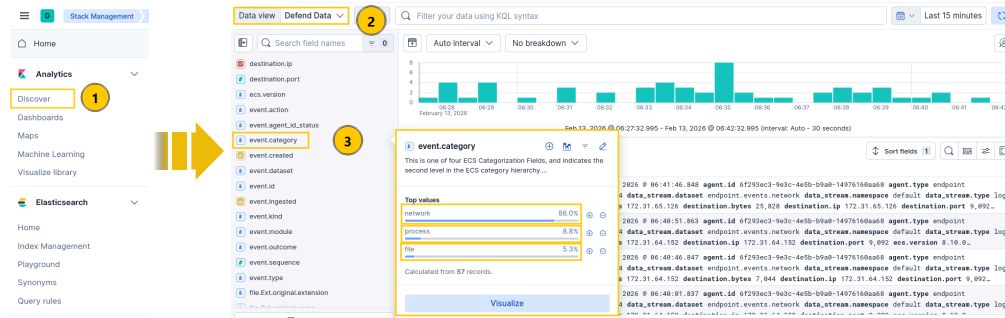
After saving the data view, let's **Set as default**.



Lastly, using the Kibana menu in the top left.

1. Select **Discover**
2. Ensure your newly made Data view is active

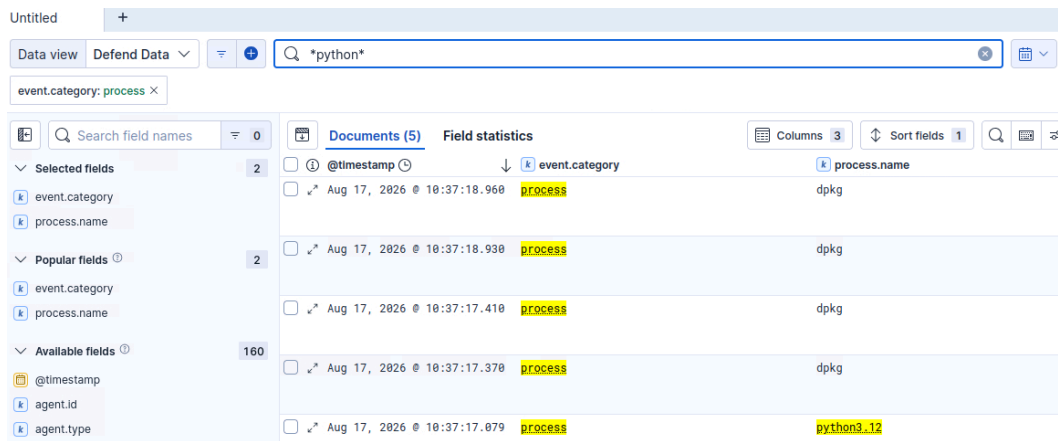
3. Investigate the `event.category` field to verify we have data coming in from our three types of monitored activity: `file`, `network`, and `process`



Answer the questions below

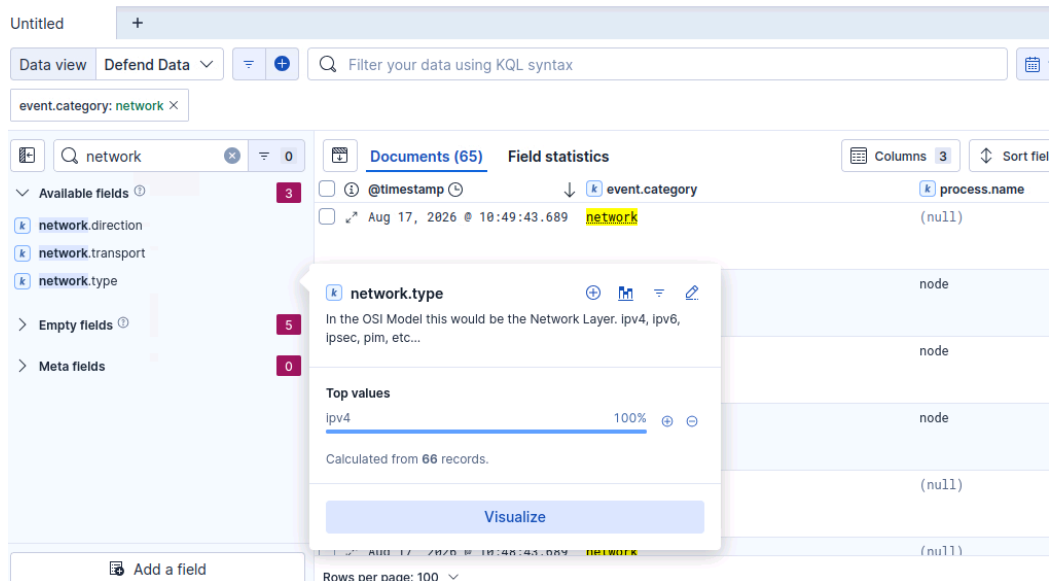
Highlight the process telemetry data being collected on your VM with `event.category: process`. What is the full `process.name` field value for the `Python` process?

python3.12



Next, highlight the network events with `event.category: network`. What is the top value for the `network.type` field?

ipv4



Task 4 - Exploring Elastic Defend Data

Enumeration via CLI: All tools used in this walkthrough are in the `/home/ubuntu/Downloads` folder.

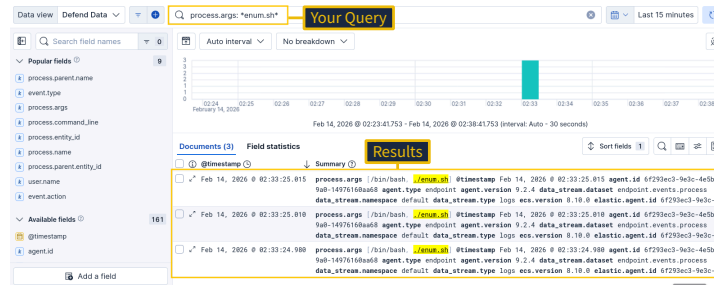
To Continue, Run the Enumeration Script

```
ubuntu@elasticdefend:~$ cd Downloads
ubuntu@elasticdefend:~/Downloads$ ./enum.sh
ubuntu
uid=1000(ubuntu) gid=1000(ubuntu) groups=1000(ubuntu),4(adm),20(dialout),24(cdrom),
25(floppy),27(sudo)
Linux elasticdefend 6.14.0-1016-aws #16~24.04.1-Ubuntu SMP Tue Oct 14 02:15:09 UTC
2025 x86_64 GNU/Linux
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
...
```

Tracking Command Line Usage: We know the script we ran is named `enum.sh`. Let's begin our investigation there. Ensure you're using the data view we set up and submit the following wildcard query:

- `process.args: *enum.sh*`

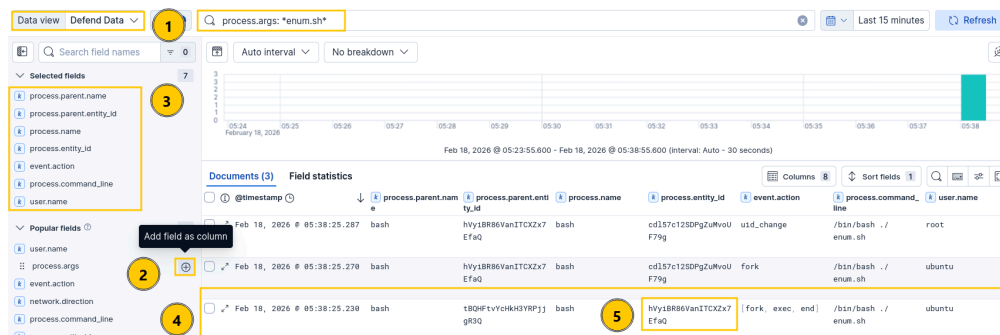
This query will allow us to highlight script execution and begin piecing together a process chain of what occurred on our machine. From the screenshot below and the results in your Elastic instance, you can see that our search returns three results.



As of now, the data is visible, but it is not very clear what's going on. Let's construct a table using helpful Elastic Defend fields to start piecing together the events that unfolded on our machine. Using the fields sidebar, construct a table with the following fields using the **+** next to each field name.

1. Ensure you're using the **Defend Data** data view and the query `process.args: *enum.sh*`
2. Use the **+** next to each field to add them to a table
3. Add the following fields: `process.parent.name`, `process.parent.entity_id`, `process.name`, `process.entity_id`, `event.action`, `process.command_line`, `user.name`
4. Locate the earliest log entry that signifies the execution of the `enum.sh` script
5. Highlight the `process.entity_id` field value for this log and copy this now

The `process.entity_id` is a unique, non-reusable identifier generated by the Elastic Agent to track a specific process instance across its entire lifecycle. Unlike standard Process IDs (PIDs), which are frequently reused by the operating system, an Entity ID is globally unique. This makes it extremely valuable for investigation and correlating parent-child relationships. Please note that `process.entity_id` in your instance will be unique and will not match the one seen below.



So if the script was executed once, why do we see three events? When Elastic Defend collects process telemetry data, it tracks different lifecycle phases of the process. Let's take a moment to look at the `event.action` field:

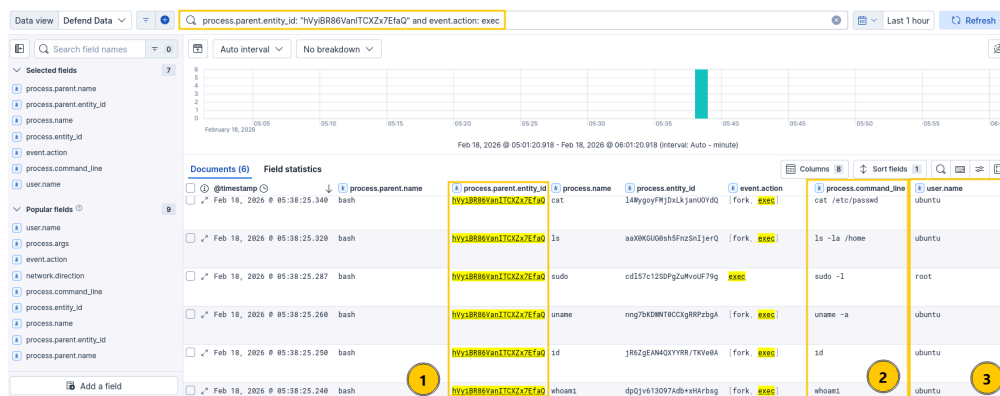
- `[fork, exec, end]` : Displays the combined lifecycle actions recorded for this specific process instance. This indicates the process was created, executed, and later terminated
- `fork` : Process creation
- `uid_change` : Process changed security context due to a `sudo` command in the script, temporarily elevating privileges to `root`

Tracking Process Chains: Now that we have the unique `process.entity_id` for our script, we can use it as a key to unlock the rest of the investigation. To see every command spawned by our script, we need to look for events where the parent process matches the process we identified. In Elastic, this means taking the `process.entity_id` you just found and querying it against the field `process.parent.entity_id`. Use the query below with the `process.entity_id` from the previous search:

- `process.parent.entity_id: "process.entity_id" and event.action: exec`

Now we have a clear view of all the commands generated from the `enum.sh` script.

1. Our unique entity ID
2. The commands executed by `enum.sh`
3. The user who executed them



This scenario demonstrates the power of pivoting through telemetry to reconstruct an adversary's actions. While we focused on process execution, remember that Elastic Defend provides additional data on file and network-related events. Let's use what we learned above to conduct a small investigation and answer the questions below. You can start by returning to the command line and running the `discovery.sh` script with the following command.

- `/home/ubuntu/Downloads/discovery.sh`

Answer the questions below

Head back to Discover and enter the query: `process.args: *discovery.sh*`. What is the `process.parent.executable` field value for the event you located?

`/usr/bin/bash`

Untitled +

Data view Defend Data + Q process.args: *discovery.sh*

Q process.parent.ex + 0

Documents (1) Field statistics

	@timestamp	process.name	process.parent.name	process.parent.entity_id	process.parent.entity_id
✓	Aug 17, 2026 @ 12:23:18.080	bash	bash	rW01VJIqIYWSv0PSpdCXQ	02PygM30x87I

process.parent.executable + 1

Absolute path to the process executable.

Top values

/usr/bin/bash 100%

Calculated from 1 record.

Multi fields

- process.parent.executable.caseless
- process.parent.executable.text

Locate the `process.entity_id` field value from the event above. Use it against the `process.parent.entity_id` field to track the process chain. What is the first command executed by the script?

find /home -name creds

Data view Defend Data + Q process.parent.entity_id: "02PygM28GVijM330x87IEg" and Aug 9, 2026 @ 00:0... → now

Q process.entity_id + 0

Documents (5) Field statistics

	@timestamp	process.name	process.parent.name	process.parent.entity_id	process.parent.entity_id	process.parent.entity_id	event.action	process.command_line	user.name
✓	Aug 17, 2026 @ 12:23:18.360	cat	bash	02PygM28GVijM330x87IEg	02PygM28GVijM330x87IEg	oYzT9BXsrPf3mHIEDB+cSg	[fork, exec]	cat /home/ubuntu/ Stuff/...	ubuntu
✓	Aug 17, 2026 @ 12:23:18.350	mkdir	bash	02PygM28GVijM330x87IEg	02PygM28GVijM330x87IEg	m8D0wwXA847JXrmz8srgfQ	[fork, exec, end]	mkdir /tmp/ creds	ubuntu
✓	Aug 17, 2026 @ 12:23:18.340	ls	bash	02PygM28GVijM330x87IEg	02PygM28GVijM330x87IEg	mf85J4MTLdJSb56PwHw1/A	[fork, exec]	ls -la /dev/ shm	ubuntu
✓	Aug 17, 2026 @ 12:23:18.330	ls	bash	02PygM28GVijM330x87IEg	02PygM28GVijM330x87IEg	bZE7CgUyPVwGw0IRjvD2LA	[fork, exec]	ls -la /temp	ubuntu
✓	Aug 17, 2026 @ 12:23:18.090	find	bash	02PygM28GVijM330x87IEg	02PygM28GVijM330x87IEg	PYz5ZC+/9r2M5dx2j0gpzQ	[fork, exec]	find /home -name *creds*	ubuntu

Continue looking through the commands that were executed. What is the name of the directory created in `/tmp`?

creds

Search the query `process.args:[*enum.sh*]`. The values `[fork, exec, end]` indicate the combined lifecycle actions recorded for this specific process instance. Take the **process parent entity ID** and use it in the following query: `process.parent.entity_id:"process.entity_id" AND event.action:exec`

Task 5 - Elastic Security Alerts and Analysis

Elastic Security alerts highlight actionable threat data and can be viewed directly in Discover because they are stored as standard documents in Elasticsearch.

Creating an Alert: Let's test out Elastic Defend's antivirus capabilities by creating a harmless test malicious document and investigating the alert in Discover. Head to the command line now and enter the following command:

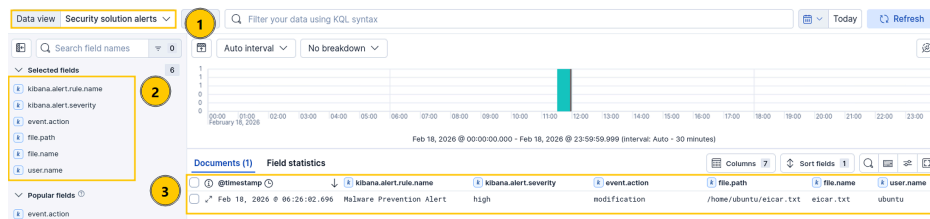
- `echo 'X50!P%@AP[4\ZX54(P^)7CC)7]$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*' > eicar.txt`

After entering the command, if you list the directory contents, you may notice that our `eicar.txt` test file doesn't exist. Earlier, when configuring the Elastic Defend integration, we ensured that prevention was enabled, so this file was quarantined immediately upon creation.

Alerts in Discover

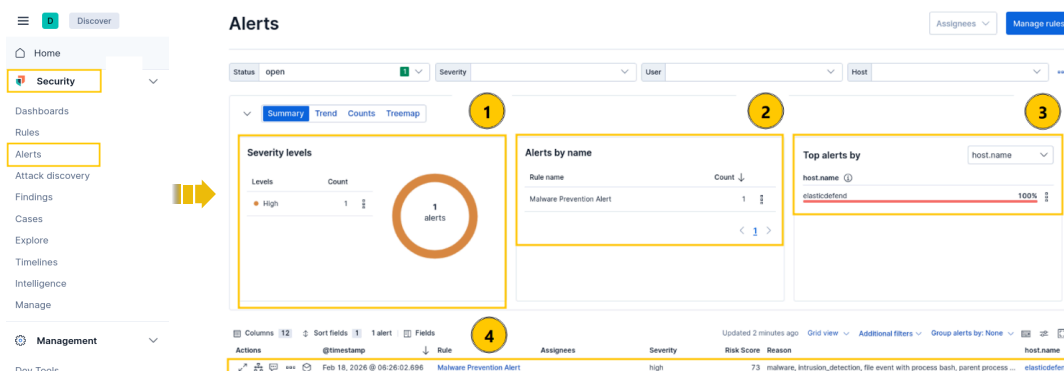
Let's head back to Discover, but this time we will use the `Security solution alerts` data view to investigate the alert.

1. Select the `Security solution alerts` data view
2. Build a table with the following fields: `kibana.alert.rule.name``kibana.alert.severity``event.action``file.path``file.name``user.name`
3. Investigate the generated alert



Elastic Security Alerts Dashboard: Using the Kibana menu, head to the `Security` section and select `Alerts`. We can now view a summary of our alert, including:

1. The severity level
2. The name of the alert
3. The host on which it triggered
4. A table containing an overview of alerts



This is a lot of information to take in, and many actions can be taken from this screen. For now, let's expand the malware alert by clicking the `View details` button.

View details	1/12	Sort fields	1 alert	Fields	Updated 6 minutes ago	Grid view	Additional filters	Group alerts by: None		
Actions	@timestamp	Rule	Severity	Risk Score	Reason	host.name	user.name	process.name		
Click to View Details	2026 @ 06:26:02.696	Malware Prevention Alert	high	73	malware, intrusion_detection, file event with process b...	elasticdefend	ubuntu	bash		

From the flyout panel, we have a wealth of information available, including descriptions of the triggered rule, relevant fields, and even suggested response actions.

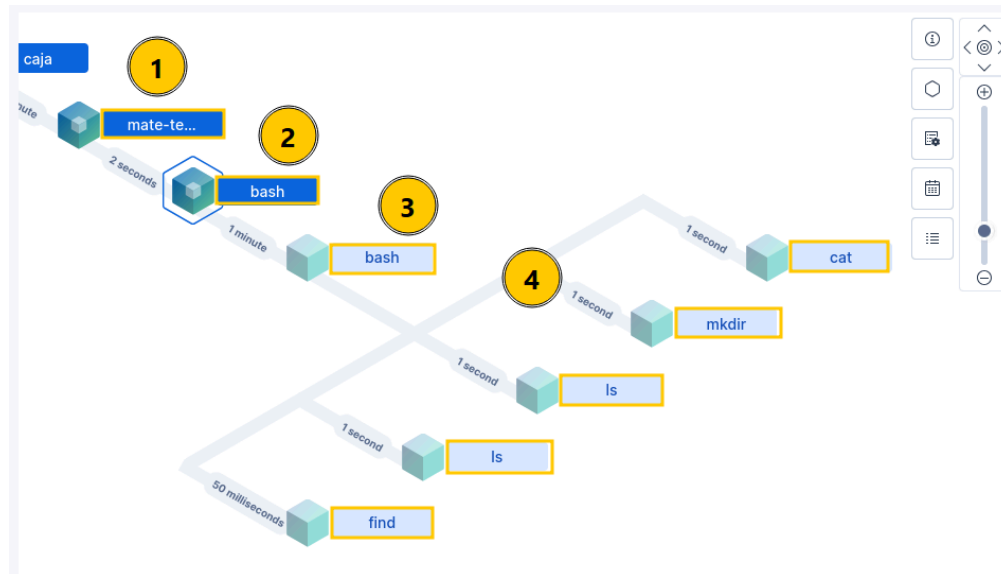
1. The **Overview** section
2. **Rule description** and **Alert reason**
3. Highlighted important fields
4. The **Table** section, which includes all fields associated with the alert. In the table section, you can pin important fields for your investigation

Analyzing the Event: The Analyzer graph visually maps parent and child process relationships over time, replacing manual correlation with a clear picture of how activity unfolded. Go ahead and select the **Analyze event** option from the malware alert in the Alerts dashboard.

Columns	1	Analyze event	1 alert	Fields	Updated 28 minutes ago	Grid view	Additional filters	Group alerts by: None		
Actions		Open the Analyzer Graph								
Severity	Risk Score	Reason	host.name	user.name	process.name	file.name				
Malware Prevention Alert	high	73	malware, intrusion_detection, file event with process b...	elasticdefend	ubuntu	bash	elastic.txt			

When you open the Analyzer graph, Elastic centers on the alert-triggering event and reconstructs the surrounding activity into a visual process tree for easier investigation. Depending on what commands you have executed in your session, your process tree may look slightly different, but in the example below, you can see:

1. The parent process
2. The process that triggered the alert
3. The child process
4. Further child processes



The EICAR example you just observed is intentionally simple. It primarily demonstrates a basic execution event where a shell launched a command that triggered a detection, allowing you to clearly see how the Analyzer graph visualizes parent and child processes. In the next task, you'll move beyond this simple execution example and begin analyzing more complex behavior patterns.

Alert Investigation: Let's put our newly gained alerting knowledge to the test by creating and investigating a new alert. Go ahead and hop back into the command line of our lab machine, ensure we are running as `root`, and attempt to read and copy the `/etc/shadow` file using the commands below. Keep in mind that some detection rules execute on a scheduled interval rather than in real time. Because of this, you may need to wait a few minutes after running the previous commands before your alert appears in the dashboard.

- `sudo su`
- `cat /etc/shadow > shadow_copy`

Answer the questions below

Investigate the malware prevention alert created from the EICAR test file. What is the `event.risk_score` field value? 73

filter event.risk_score

What `event.code` field value is assigned to the above alert? malicious_file

filter event.code

What is the risk score assigned to the `/etc/shadow` file read alert? 47

run the query in terminal `sudo su` then `cat /etc/shadow > shadow_copy`

then go to the alert, click potential shadow copy, then click view details

Using the flyout menu for the `/etc/shadow` file read alert, investigate the Overview section. What is the first MITRE ATT&CK tactic name associated with the alert?

Privilege Escalation

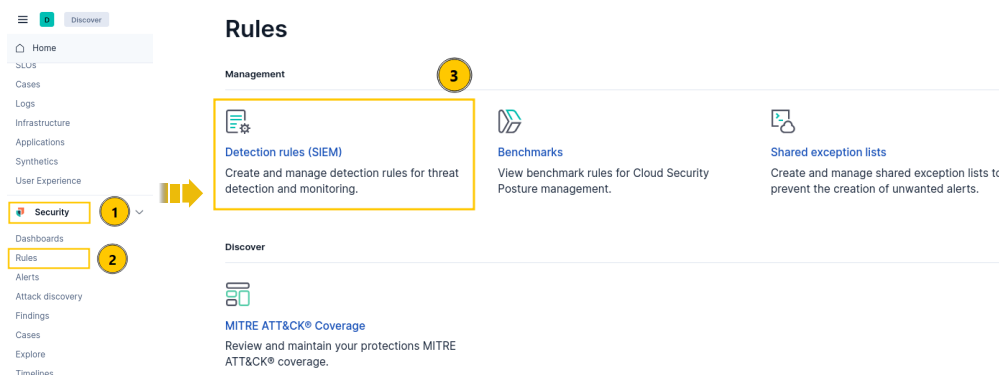
Go to the alert, click **Potential Shadow Copy**, and then click **Overview Details**.

Task 6 - Detection Rules and Further Investigation

Elastic Defend's prebuilt protection, Elastic Security provides diverse custom detection rules—including Custom Query, Threshold, Event Correlation (EQL), Machine Learning, Indicator Match, New Terms, and ES|QL rules—allowing you to tailor threat detection across your entire environment.

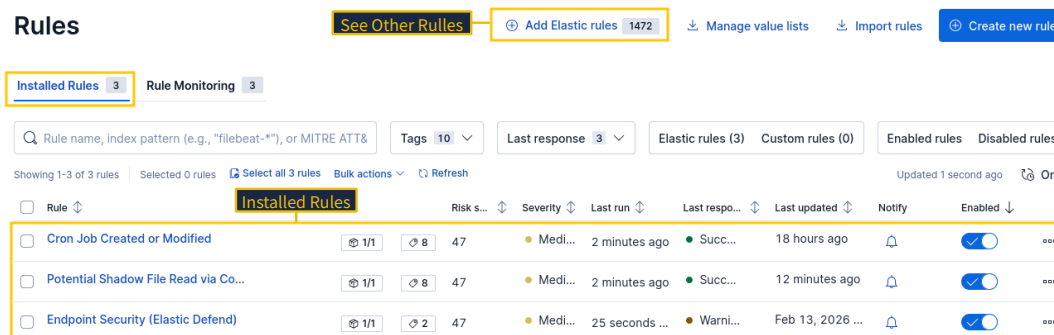
Detection Rules: Before triggering our next alert, let's see how we can manage detection rules.

1. Using the Kibana menu, head to **Security**
2. Select **Rules**
3. Click **Detection rules (SIEM)**



From here, we can see a list of our installed rules, including Elastic Defend's native rule list and two pre-added rules. You may notice that the **Potential Shadow File Read via Command Line Utilities** rule is added here, which we triggered and investigated in the previous task.

- Click **+ Add Elastic rules** to view Elastic's extensive rule list.



On the next screen, you can search and enable any rules you like based on your specific needs. For now, let's keep the rules we have enabled and head back to the command line.

Add Elastic rules

[Install all](#)

Tags 185

☐ Rule	Risk score	Severity	
☐ Potential Internal Linux SSH Brute Force Detected	1/1 5 47	Medium	Install
☐ Potential Linux Hack Tool Launched	1/4 10 47	Medium	Install
☐ Host File System Changes via Windows Subsystem for Linux	1/3 9 47	Medium	Install
☐ Potential Linux Backdoor User Account Creation	1/4 10 47	Medium	Install

Triggering an Alert: Let's hop back into the command line of our lab machine, ensure we are running as root, and run the last script with the commands below to trigger an alert for the rules we added.

- `sudo su`
- `/home/ubuntu/Downloads/persist.sh`

This script will simulate a persistence technique by creating a cron job that repeatedly attempts to establish a reverse connection to a specified IP address.

Investigating the Alert: Go ahead and hop back into the Alerts dashboard in Kibana. Remember that the rules run on a schedule, so you may need to wait a moment for the alert to appear. In the screenshot below, and in your VM instance, you will see our third alert titled `Cron Job Created or Modified`. Let's start in the `View details` flyout panel.

The screenshot shows the Kibana Alerts dashboard. On the left, there's a 'Severity levels' section with a donut chart showing 3 alerts (1 High, 2 Medium). Below it, a table lists alerts with columns for Actions, @timestamp, Rule, and Assignees. One alert is highlighted with a yellow box and a label 'Click to View Details'. On the right, the 'Alert Details' flyout is open for the 'Cron Job Created or Modified' alert. It shows the alert status as 'Open', risk score as 47, and a description of the rule. The 'About' section includes the rule description and alert reason. The 'Table' tab is selected, showing the alert details in a table format.

At this point, we want to try to answer some basic questions about the alert. Let's use the data available to us to develop a basic understanding of what occurred. Again, in the Table tab of the flyout menu, you can pin important fields. Let's give it a shot by pinning the following fields.

- `@timestamp`
- `file.name`
- `file.path`
- `process.name`
- `user.name`

From the following information, we can start to build an idea of what occurred and whether this behavior is indeed malicious.

Medium

Feb 15, 2026 @ 11:55:33.779

Cron Job Created or Modified

Status

Open

Risk score

47

Assignees

Notes

Add note

Overview

Table

JSON

Filter by field or value...

Click to Pin a Field

Value

Your Pinned Fields

@timestamp

Feb 15, 2026 @ 11:55:33.779

file.name

system-update

file.path

/etc/cron.d/system-update

process.name

bash

user.name

root

In the previous task, we explored the analyzer graph to investigate the malware alert. Now, we will delve further to gain a deeper understanding of the events that unfolded when we ran the last script. Go ahead and open the analyzer graph for the **Cron Job Created or Modified** alert.

Summary

Trend

Counts

Treemap

Severity levels

Levels	Count
High	1
Medium	2

3 alerts

Alerts by name

Rule name	Count
Cron Job Created or Modified	1
Malware Prevention Alert	1
Potential Shadow File Read via Comma...	1

Top alerts by

host.name

elasticdefend

100%

Columns 12

Analyze event

Open the Analyzer Graph

Updated 1 hour ago

Grid view

Additional filters

Group alerts by: None

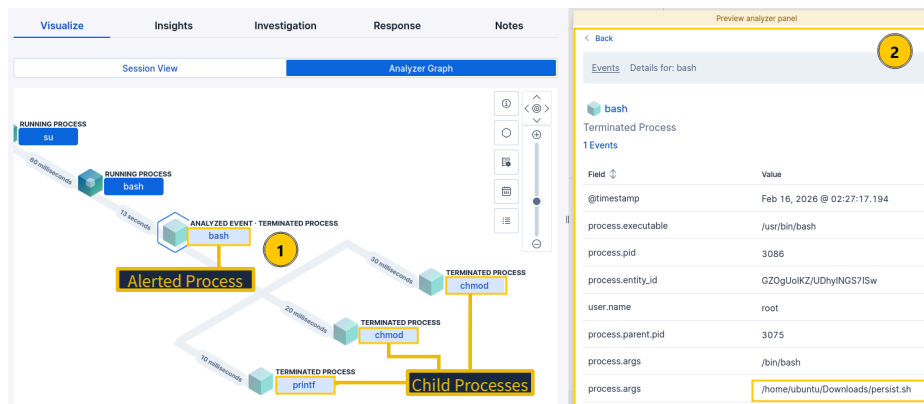
Assignees	Severity	Risk Score	Reason
Cron Job Created or Modified	medium	47	file event with process bash, file system-updi
Potential Shadow File Read via Com...	medium	47	process event with process cat, parent proce
Malware Prevention Alert	high	73	malware, intrusion_detection, file event with f

L2 - 3.3. Elastic: Using Elastic Defend

15

We know from the previous task that the analyzer graph is automatically centered on the target process and provides a visual representation of the entire process chain. Let's investigate further!

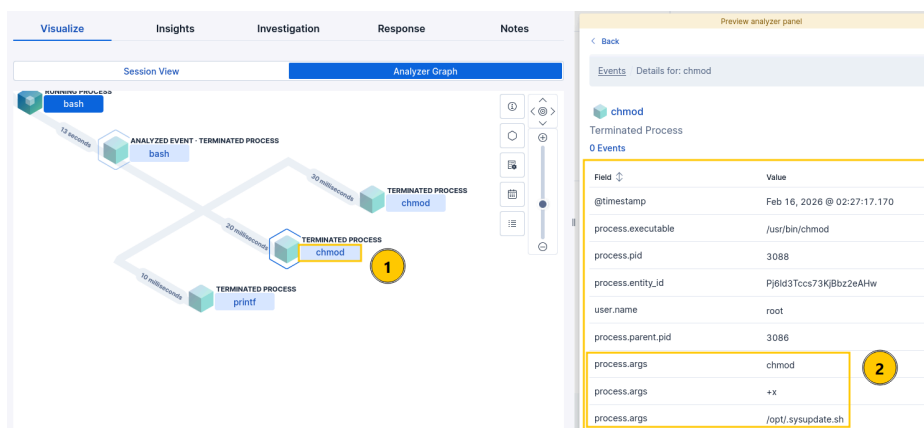
1. Click the `bash` process that triggered the alert
2. Investigate the overview flyout menu



Investigating Child Processes: The script we executed clearly established persistence, but assume we don't have access to the script itself. As analysts, we must rely on endpoint telemetry to determine what actions were performed. To begin reconstructing the activity, let's examine the first `chmod` process spawned by the script:

1. Click the `chmod` process
2. View the event details

Looking at the `process.args` field values, you can see that execution permissions were set for a bash script named `.sysupdate.sh` using the command `chmod +x /opt/.sysupdate`. This tells us that a script named `.sysupdate.sh` exists in `/opt`, it was made executable, and the action was performed by a child process of the original `bash` execution that was alerted on.

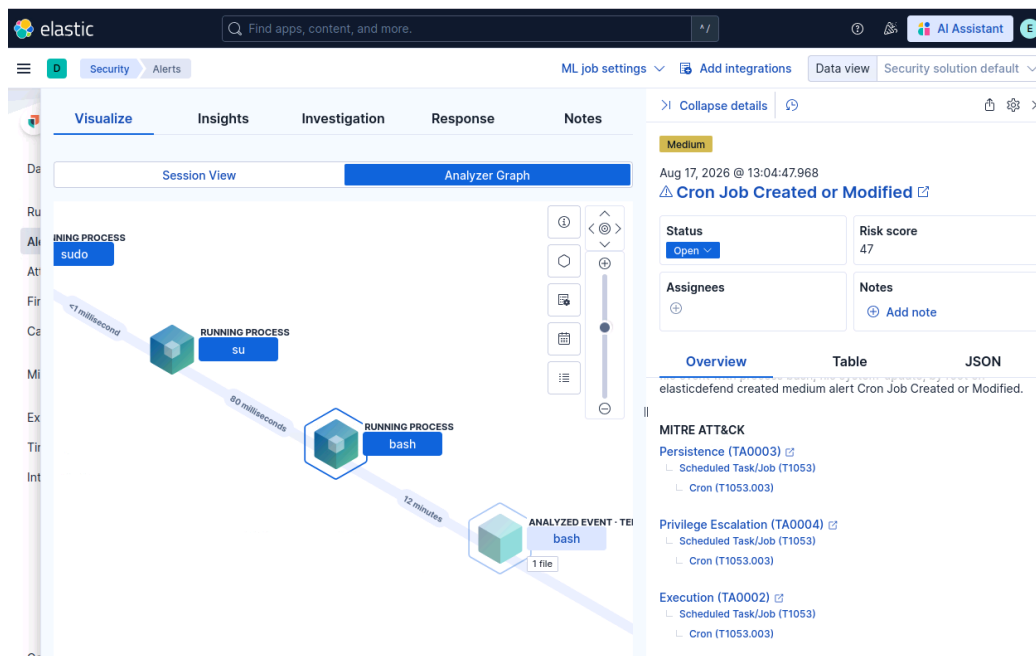


Investigation: By examining parent and child process relationships through the Analyzer graph and event flyouts, you can map the complete attack sequence and understand the full scope of system activity.

Answer the questions below

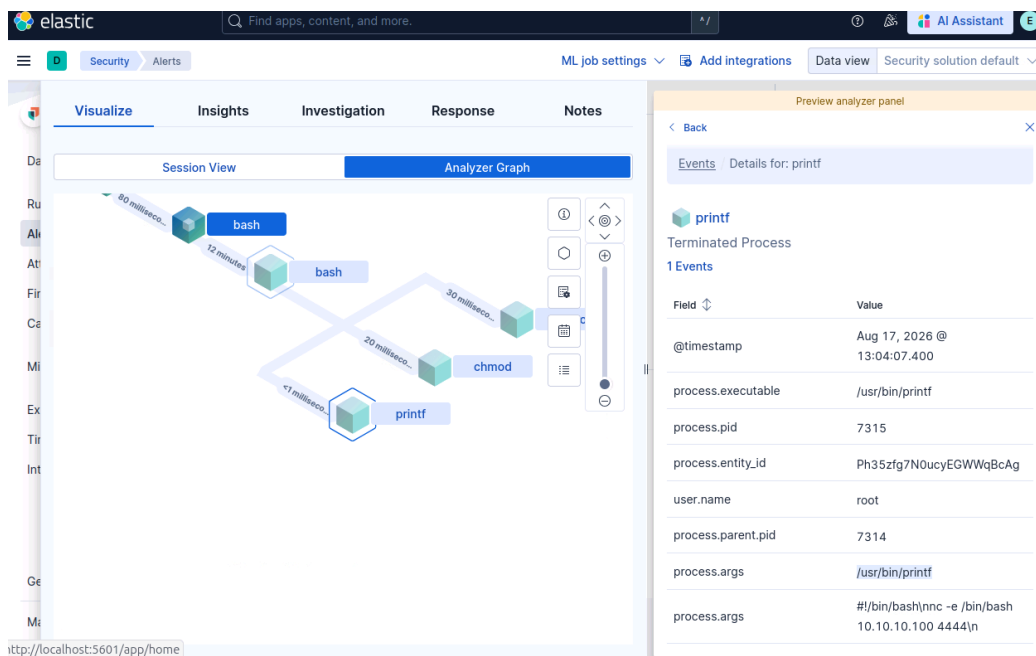
How many MITRE Tactics are associated with the `Cron Job Created or Modified` alert?

3



Investigate the `printf` child process using the Analyzer graph and flyout panel. What is the `process.executable` field value?

`/usr/bin/printf`



Which IP address and port number were used for the reverse shell attempt?

10.10.100.4444

Check out the remaining `chmod` child process. What is the name of the cron job whose permissions were set?

system-update

Task 7 - Conclusion

In this room, you configured the Elastic Defend integration and set up a custom Kibana data view to capture and isolate endpoint activity. By executing commands on the host, you generated live telemetry, which you then investigated through the Alerts dashboard and Visual Event Analyzer to map parent-child process chains and reconstruct potential attacks.